

Design a Meeting Scheduler

Uber • Salesforce • Google Calendar • Outlook – ఇది సులభం అనిపిస్తుంది, మరియు అందుకే మొదటి విరుపు ఒక్క అక్షరం నుంచి వస్తుంది.

ఇక్కడ మూడు విరుపులు ఉన్నాయి. మొదటిది ఒక సరిహద్దు bug – వెనువెంటనే జరిగే రెండు meetings ఢీ కొట్టినట్టు చూపిస్తుంది. రెండోది ఒక ఖాళీ కిటికీని పూర్తిగా మిస్ చేస్తుంది. మూడోది ఒక standup ని 52 rows గా మారుస్తుంది.

ఈ doc ఎలా చదవాలి

పక్కన editor తెరిచి కలిసి రాయండి. ఇక్కడి ప్రతి output నిజంగా `node lo run` చేసినదే.

ఈ PROBLEM నేర్చుకుంటే - చాలా వస్తాయి

Meeting room booking, hotel rooms, doctor appointments, flight schedules, resource allocation, cron jobs - ఇవన్నీ ఒకే గణిత సమస్య: కాల విరామాలు, అవి ఢీ కొట్టడం, మరియు ఖాళీలు వెతకడం.

మరియు ఈ doc lo నేర్చుకునే మొదటి విషయం - అర్థ-తెరిచిన విరామాలు (§6) - ఒక సార్వత్రిక సాధనం. ఒకసారి అది అలవాటైతే, మీరు ఇక ఎప్పుడూ ఆ సరిహద్దు bug రాయరు.

విషయ సూచిక (Table of Contents)

Part 1 - సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి - మరియు మూడు వేర్వేరు ప్రశ్నలు
2. Clarifying questions - ప్రతిదాని వెనక ఉన్న ఉద్దేశం

Part 2 - మొదటి విరుపు: ఒక అక్షరం

3. Step - bookings ఒక జాబితా
4. మొదటి విరుపు - 11:00 కి ముగిసేది 11:00 కి మొదలయ్యేదాన్ని ఆపింది
5. Step - అర్థ-తెరిచిన విరామాలు

Part 3 - రెండో విరుపు: grid మీద వెతకడం

6. Step - "ప్రతి 15 నిమిషాలకీ చూద్దాం"
7. రెండో విరుపు - నకిలీ slots, మరియు మిస్ అయిన ఖాళీ
8. Step - Busy ని కలిపి, ఖాళీలని తీయడం

Part 4 - మూడో విరుపు: పునరావృత్తి

9. మూడో విరుపు - ఒక standup, 52 rows
10. Step - నియమాన్ని store చేయడం
11. మినహాయింపులు - "ఈ ఒక్క వారం వద్దు"

Part 5 - పూర్తి system

12. Step - ఢీ కొంటే ఏం చేయాలి · ఒక మారే నియమం
13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

Part 6 - Interview lo

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

15. నోటిస్ చెప్పాల్సిన English script
16. Follow-ups – timezones, scale, notifications
17. ఏమి నేర్చుకున్నాం

Part 1 – సమస్యని నిజంగా అర్థం చేసుకోవడం

1. అడిగింది ఏమిటి – మరియు మూడు వేర్వేరు ప్రశ్నలు

"Design a meeting scheduler. Users book rooms for meetings, and the system should prevent double-booking and suggest free slots."

ఇందులో మూడు వేర్వేరు ప్రశ్నలు ఉన్నాయి, మరియు అవి మూడు వేర్వేరు కష్టాలు:

మూడు ప్రశ్నలు • మూడు వేర్వేరు కష్టాలు

"Book a room" – లోపల ఏమి ఉంది

1 • "ఇవి ఢీ కొడతాయా?"

రెండు విరామాలు కలుస్తాయా
సులభం అనిపిస్తుంది –
కానీ సరిహద్దు వద్ద తప్పు
→ §4 • ఒక్క అక్షరం bug

2 • "ఎప్పుడు ఖాళీ?"

ఇది తిరగబడిన ప్రశ్న –
busy నుంచి free ని తీయడం
పలు attendees అయితే మరింత కష్టం
→ §7 • ఇదే అసలు కష్టం

3 • "ప్రతి సోమవారం"

పునరావృత్త meetings
ఇది ఒక booking కాదు –
ఇది ఒక నియమం
→ §9 • storage ప్రశ్న

ఎక్కడ ఎవరు ఆగిపోతారు

ప్రశ్న 1 ని అందరూ రాస్తారు – కానీ చాలా మంది తప్పుగా రాస్తారు, మరియు గమనించరు
ప్రశ్న 2 ని చాలా మంది "ప్రతి 15 నిమిషాలకీ మార్డం" అని పరిష్కరిస్తారు – అది రెండు విధాలుగా తప్పు
ప్రశ్న 3 ని అడిగితేనే ఆలోచిస్తారు, మరియు అప్పటికి storage design ఖరారైపోయింది
మూడింటినీ మొదటి ఐదు నిమిషాల్లో పేరుపెడితే – మిగతాది సులభం.

2. Clarifying questions – ప్రతిదాని వెనక ఉన్న ఉద్దేశం

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
వెనువెంటనే meetings (10-11, 11-12) అనుమతించాలా?	అవును అనేది సహజ జవాబు – మరియు అదే §4 లో విరిగేది
పునరావృత్త meetings కావాలా?	అవును అంటే storage పూర్తిగా మారుతుంది (§9). ముందే అడగాలి
పలు attendees ఉంటే అందరి ఖాళీ కావాలా?	అవును అంటే §8 – intersection లెక్కించాలి

ప్రశ్న	జవాబు నా design ని ఎలా మారుస్తుంది
గది నిండితే ఏం చేయాలి – తిరస్కరించాలా, సూచించాలా?	ఇది ఒక మారే business నియమం (§12)
Timezones?	ఇది మరే లేవనెత్తాలి – §16
Buffer time (meetings మధ్య 10 నిమి)?	ఉంటే interval ని పెంచి తనిఖీ చేయడం
ఎన్ని rooms, ఎన్ని bookings?	Data structure ఎంపికని నిర్ణయిస్తుంది

మొదటి ప్రశ్న ఒక ఉచ్చు, మరియు చాలా మంది దాన్లో పడతారు

"10:00-11:00 మరియు 11:00-12:00 – ఇవి ఢీ కొడతాయా?"

జవాబు స్పష్టంగా **లేదు**. మొదటిది 11:00 కి ముగుస్తుంది, రెండోది 11:00 కి మొదలవుతుంది. ఒక్క సెకను కూడా కలవట్లేదు. ప్రతి ఆఫీసులో రోజూ ఇలాగే జరుగుతుంది.

కానీ మీరు overlap ని **సహజంగా** రాస్తే – `aStart <= bEnd && aEnd >= bStart` – అది **అవును** అని చెప్తుంది. §4 lo దాన్ని నడిపి చూద్దాం.

Part 2 – మొదటి విరుపు: ఒక అక్షరం

3. Step – bookings ఒక జాబితా

అతి సహజమైన మొదటి version. ఒక గది, ఒక bookings జాబితా, మరియు overlap తనిఖీ:

```
class Room {
  constructor(id) { this.id = id; this.bookings = []; }

  overlaps(aS, aE, bS, bE) { return aS <= bE && aE >= bS; }    // ← "సహజమైన" తనిఖీ

  book(start, end, title) {
    const hits = (b) => this.overlaps(start, end, b.start, b.end);
    const clash = this.bookings.find(hits);
    if (clash) return { ok: false, reason: `CLASH: ${clash.title}` };
    this.bookings.push({ start, end, title });
    return { ok: true };
  }
}
```

ఆ `overlaps` ఆలోచన సరళమైనది: "A మొదలు B ముగింపు కంటే ముందు ఉంది, మరియు A ముగింపు B మొదలు కంటే తర్వాత ఉంది → కలుస్తున్నాయి." ఇది చదవడానికి సరైనదిగానే అనిపిస్తుంది.

4. మొదటి విరుపు – 11:00 కి ముగిసేది 11:00 కి మొదలయ్యేదాన్ని ఆపింది

ఒక సాధారణ ఉదయం: 10 గంటలకి standup, ఆపై 11 గంటలకి design చర్చ.

```
const r = new Room('Everest');
console.log('10:00–11:00 standup :', r.book(t('10:00'), t('11:00'), 'standup'));
console.log('11:00–12:00 design  :', r.book(t('11:00'), t('12:00'), 'design'));
```

```
10:00–11:00 standup : { ok: true }
11:00–12:00 design  : { ok: false, reason: 'CLASH: standup' }

bookings: [ '10:00–11:00 standup' ]
```

మొదటి విరుపు - వెనువెంటనే MEETINGS అసాధ్యమయ్యాయి

Design చర్చ తీరస్కరించబడింది, ఎందుకంటే standup 11:00 కి ముగుస్తోంది మరియు ఇది 11:00 కి మొదలవుతోంది.

ఇది ఒక అరుదైన edge case కాదు - **ప్రతి ఆఫీసులో రోజూ జరిగేది**. మీ calendar app ఇలా ప్రవర్తిస్తే అది మొదటి రోజే పనికిరాదని తేలిపోతుంది.

తప్పు ఎక్కడ? `aS <= bE` లో `అ` = . `11:00 <= 11:00` → నిజం. అంటే **తాకడాన్ని కలవడంగా** లెక్కిస్తున్నాం.

అసలు సమస్య: 11:00 అంటే ఏమిటి?

ఈ bug ఒక typo కాదు - ఇది ఒక **భావనాపరమైన అస్పష్టత**.

Standup "11:00 వరకు" అంటే - **11:00 ఆ meeting లో భాగమా, కాదా?**

- అది **భాగం** అనుకుంటే (మూసిన విరామం `[10:00, 11:00]`) - అప్పుడు 11:00 కి మొదలయ్యే meeting నిజంగా ఢీ కొడుతుంది.
- అది **భాగం కాదు** అనుకుంటే (అర్థ-తెరిచిన విరామం `[10:00, 11:00)`) - అప్పుడు ఢీ లేదు.

నిజ ప్రపంచంలో **రెండేదే సరైనది**. Meeting 11:00 కి ముగుస్తుంది, అంటే 10:59:59 వరకు నడుస్తుంది. 11:00 అనేది ఒక **సరిహద్దు**, ఒక క్షణం కాదు.

5. Step - అర్థ-తెరిచిన విరామాలు

ఆ నియమాన్ని ఒక class లో పెడదాం, మరియు overlap logic ని **ఒక్కచోట** ఉంచుదాం:

```
class Interval {
  constructor(start, end) {
    if (end <= start) throw new Error(`EMPTY_INTERVAL: ${hhmm(start)}-${hhmm(end)}`);
    Object.assign(this, { start, end }); Object.freeze(this);
  }
  overlaps(o) { return this.start < o.end && o.start < this.end; } // ← కచ్చితంగా '<'
  get minutes() { return this.end - this.start; }
}
```

`<=` రెండూ `<` అయ్యాయి. **అంతే మార్పు**. మరియు constructor లో ఒక తనిఖీ - ఖాళీ విరామం (`end <= start`) అర్థరహితం, కాబట్టి అది ఎప్పుడూ సృష్టించబడకూడదు.

ఐదు సందర్భాలు పరీక్షిద్దాం:

10:00-11:00 vs 11:00-12:00	→ సరే ✓	(వెనువెంటనే - ఢీ కొట్టకూడదు)
10:00-11:00 vs 10:30-11:30	→ ఢీ ✗	(పాక్షికంగా కలుస్తాయి)
10:00-12:00 vs 10:30-11:00	→ ఢీ ✗	(ఒకదానిలో ఒకటి)
10:00-11:00 vs 09:00-10:00	→ సరే ✓	(ముందు ముగిసింది)
10:00-11:00 vs 11:01-12:00	→ సరే ✓	(ఒక నిమిషం ఖాళీ)

ఖాళీ విరామం:

EMPTY_INTERVAL: 10:00-10:00

ఐదూ సరైనవి, మరియు ఖాళీ విరామం తిరస్కరించబడింది.

ఈ ఒక్క **CLASS** చాలా మంది మీస్ చేసే ఒక **DESIGN** విజయం

చాలా మంది `overlaps()` ని `Meeting` class లో రాస్తారు. అప్పుడు ఏమవుతుంది? – అదే logic `Room` లో, `User` లో, `Resource` లో **నకలు** అవుతుంది. మరియు ఆ `<=` bug ని ఒక చోట సరిచేస్తే మిగతా చోట్ల అలాగే ఉండిపోతుంది.

Overlap అనేది విరామాల లక్షణం, meetings యొక్క కాదు. ఒక `Interval` విలువ వస్తువు రాసి అందరూ దాన్ని వాడటం – ఇది చిన్న నిర్ణయంలా కనిపిస్తుంది, కానీ interviewer దీన్ని గమనిస్తాడు.

Part 3 – రెండో విరుపు: grid మీద వెతకడం

6. Step – "ప్రతి 15 నిమిషాలకీ చూద్దాం"

ఇప్పుడు రెండో ప్రశ్న: "ముగ్గురికీ ఉమ్మడిగా 30 నిమిషాలు ఎప్పుడు ఖాళీ?"

సహజమైన ఆలోచన: రోజంతా 15-నిమిషాల అడుగులతో నడిచి, ప్రతి చోటా అందరూ ఖాళీయేనా చూడటం.

```
const overlaps = (a, b) => a.start < b.end && b.start < a.end; // §5 నియమమే

function naiveFreeSlots(calendars, durationMin, dayStart, dayEnd, step = 15) {
  const out = [];
  for (let s = dayStart; s + durationMin <= dayEnd; s += step) {
    const slot = { start: s, end: s + durationMin };
    if (!calendars.some(c => c.some(m => overlaps(slot, m)))) out.push(slot);
  }
  return out;
}
```

ఇది పనిచేస్తుంది, మరియు రాయడానికి సులభం:

```
ముగ్గురికీ ఉమ్మడిగా ఖాళీ (30 నిమిషాలు):
10:30–11:00  10:45–11:15  11:00–11:30  15:00–15:30  15:15–15:45  15:30–16:00 ... మొత్తం
9
```

తొమ్మిది slots దొరికాయి. **జవాబు సరైనదే** – ఆ ప్రతి slot lo నిజంగా అందరూ ఖాళీగా ఉన్నారు.

మరి సమస్య ఏమిటి?

7. రెండో విరుపు – సకిల్ slots, మరియు మిస్ అయిన ఖాళీ

ఆ output ని మళ్ళీ చూడండి. 10:30–11:00, 10:45–11:15, 11:00–11:30 – ఇవి మూడు వేర్వేరు slots కాదు. ఇవి ఒకే ఖాళీ కిటికీ 10:30–11:30 ని మూడు విధాలుగా చూపిస్తున్నాయి.

దాన్ని విడిగా చూద్దాం:

```
సమస్య 1 – ఒకే ఖాళీ కిటికీ, చాలా "slots"
10:30–11:00  10:45–11:15  11:00–11:30
→ నిజానికి ఖాళీ ఒక్కటే: 10:30–11:30. కానీ 3 "slots" చూపిస్తోంది, అవి ఒకదానితో ఒకటి కలుస్తున్నాయి.

సమస్య 2 – నిజమైన ఖాళీని పూర్తిగా మిస్ చేయడం
15-నిమిషాల grid తో దొరికినవి: (ఏమీ లేవు)
```

- 10:07-10:37 lo సరిగ్గా 30 నిమిషాలు ఖాళీ ఉంది, కానీ అది grid మీద పడలేదు.
- step=1 చేస్తే దొరుకుతుంది, కానీ పని 15 రెట్లు పెరుగుతుంది.

రెండో విరుపు - GRID ఒక అంచనా, ఒక జవాబు కాదు

సమస్య 1: User కి ఒక ఖాళీ కిటికీ చూపించాలి, మూడు overlapping సూచనలు కాదు. UI lo ఇది గందరగోళం.

సమస్య 2 చాలా ఫోరం: 10:07 నుంచి 10:37 వరకు సరిగ్గా 30 నిమిషాలు ఖాళీ ఉంది. కానీ 15-నిమిషాల grid ఆ ప్రారంభాన్ని ఎప్పుడూ తాకదు, కాబట్టి system "ఏ ఖాళీ లేదు" అని చెప్తుంది. **నిజమైన, ఉపయోగపడే slot ని పూర్తిగా మాయం చేసింది.**

Step ని 1 నిమిషం చేస్తే దొరుకుతుంది - కానీ పని 15 రెట్లు పెరుగుతుంది, మరియు సమస్య 1 **15 రెట్లు ఫోరమవుతుంది** (ఇప్పుడు ఒకే కిటికీకి 30 slots).

మౌలిక తప్పు: మనం ఖాళీని లెక్కించలేదు, దాని కోసం వెతుకుతున్నాం - మరియు కొన్ని చోట్ల మాత్రమే చూస్తున్నాం.

8. Step - Busy ని కలిపి, ఖాళీలని తీయడం

ప్రశ్నని తిప్పుదాం. "ఖాళీ ఎక్కడ?" అని వెతకడానికి బదులు - **"busy ఎక్కడ?" అని తీసుకుని, మిగిలినదే ఖాళీ.**

మూడు అడుగులు: (1) అందరి busy విరామాలని ఒకే జాబితాగా చేసి, (2) క్రమంలో పెట్టి కలిపేసి, (3) వాటి మధ్య ఖాళీలని తీయడం.

```
function freeWindows(calendars, dayStart, dayEnd) {
  const busy = calendars.flat()
    .filter(m => m.end > dayStart && m.start < dayEnd)
    .sort((a, b) => a.start - b.start); // 1 • క్రమంలో

  const merged = [];
  for (const m of busy) { // 2 • కలిపేయడం
    const last = merged[merged.length - 1];
    if (last && m.start <= last.end) last.end = Math.max(last.end, m.end);
    else merged.push({ start: Math.max(m.start, dayStart),
      end: Math.min(m.end, dayEnd) });
  }

  const free = []; let cursor = dayStart; // 3 • ఖాళీలు
  for (const m of merged) {
    if (m.start > cursor) free.push({ start: cursor, end: m.start });
    cursor = Math.max(cursor, m.end);
  }
  if (cursor < dayEnd) free.push({ start: cursor, end: dayEnd });
  return free;
}
```

ఇక్కడ ఒక సూక్ష్మమైన వివరం: merge అడుగులో `m.start <= last.end` - ఇక్కడ `<=` **సరైనదే**, §5 lo మనం తీసేసినా! ఎందుకంటే ఇది overlap తనిఖీ కాదు; ఇది "ఈ రెండు busy ముక్కలు తాకుతున్నాయా, అయితే ఒకటిగా చేద్దాం" అని అడుగుతోంది. 10-11 మరియు 11-12 busy అయితే, మధ్యలో ఖాళీ లేదు -

కాబట్టి అవి ఒకే busy ముక్క 10-12.

ఒకే గుర్తు, రెండు వేర్వేరు అర్థాలు. ఇలాంటి చోట్లే bugs దాక్కుంటాయి, మరియు ఇది గుర్తించి చెప్పడం interview lo విలువైనది.

రెండు సమస్యలనే మళ్ళీ నడుపుదాం:

సమస్య 1 తిరిగి:

10:30-11:30 ← ఒకే కిటికీ, ఒకే సారి

సమస్య 2 తిరిగి:

10:07-10:37 (30 నిమిషం) ← grid మీద లేకపోయినా దొరికింది

ముగ్గురి ఉమ్మడి ఖాళీ కిటికీలు:

10:30-11:30 15:00-16:00 17:00-18:00

30 నిమిషాలు పట్టేవి: 10:30-11:30 15:00-16:00 17:00-18:00

తొమ్మిది గందరగోళ slots → మూడు స్పష్టమైన కిటికీలు. మరియు grid మీద పడని 10:07 ఖాళీ కూడా దొరికింది.

మరియు గమనించండి – ఇది **ఎంత నిడివి కావాలో అడగకుండానే** పనిచేస్తుంది. కిటికీలు లెక్కించాక, "30 నిమిషాలు పట్టేవి ఏవి?" అనేది ఒక సాధారణ filter.

Part 4 – మూడో విరుపు: పునరావృత్తి

9. మూడో విరుపు – ఒక standup, 52 rows

Interviewer అంటాడు: "ఇప్పుడు ప్రతి సోమవారం జరిగే standup చేర్చండి."

సహజమైన ఆలోచన: **ప్రతి సందర్భాన్ని ఒక booking గా చేర్చడం.**

ఎంపిక A – అన్నిటినీ విడివిడిగా store చేయడం:

rows: 52

"ప్రతి standup ని 10:15 కి జరుపు" అంటే → 52 rows update చేయాలి

"రెండేళ్ళు" అంటే → 104 rows

"ఎప్పటికీ" అంటే → ఎన్ని rows?

మూడో విరుపు – "ఎప్పటికీ" అనే పదానికి ROWS లేవు

ఒక ఏడాదికి 52 rows పర్వాలేదు అనిపించొచ్చు. కానీ మూడు ప్రశ్నలు అడగండి:

1 • "సమయం మార్పు" – 52 rows update. ఏదైనా ఒకటి తప్పిపోతే calendar అస్థిరమవుతుంది.

2 • "ఎప్పటికీ జరిగేది" – Google Calendar లో ఇది సాధారణం. ఎన్ని rows సృష్టించాలి? 1000? 10,000? ఎక్కడ ఆపాలి?

3 • "ఇప్పటినుంచి ప్రతి రెండే మంగళవారం" – ఈ నమూనాని 52 విడి rows చూసి తిరిగి తెలుసుకోవడం అసాధ్యం. నమూనా పోయింది.

మౌలిక తప్పు: మనం ఒక నియమాన్ని దాని ఫలితాలుగా store చేస్తున్నాం. ఇది Deep Dive 05 (Splitwise) §1 లో చూసిన అదే తప్పు – అక్కడ అప్పులని store చేశాం, ఖర్చులని కాదు.

10. Step – నియమాన్ని store చేయడం

నియమాన్ని store చేసి, సందర్భాలని అడిగినప్పుడు లెక్కించడం:

```
class Recurrence {
  constructor({ freq='WEEKLY', interval=1, byDay=[1],
              until=null, count=null } = {}) {
    Object.assign(this, { freq, interval, byDay, until, count });
  }
  // ఒక పరిధిలో ఎప్పుడెప్పుడు జరుగుతుందో *లెక్కించడం*
  occurrencesIn(startDay, endDay, baseDay) {
    const out = [];
    for (let day = baseDay; day <= endDay; day += 7 * this.interval) {
      if (day < startDay) continue;
      if (this.until !== null && day > this.until) break;
      if (this.count !== null && out.length >= this.count) break;
    }
  }
}
```

```

        out.push(day);
    }
    return out;
}
}

```

ఎంపిక B – నియమాన్ని store చేయడం:
 rows: 1 (ఒక నియమం)
 ఈ వారంలో ఎప్పుడు? → రోజు 0
 రోజులు 14–35 లో? → రోజు 14, రోజు 21, రోజు 28, రోజు 35
 "10:15 కి జరుపు" అంటే → 1 row update

52 rows → 1 row. మరియు "ఎప్పటికీ" అనేది ఇప్పుడు సహజంగా వ్యక్తపరచగలం – `until: null`. అడిగిన పరిధికి మాత్రమే లెక్కిస్తాం.

ఈ ఆకారం యాదృచ్ఛికం కాదు – `freq`, `interval`, `byDay`, `until`, `count` అనేవి **iCalendar RRULE** ప్రమాణం నుంచి. Google Calendar, Outlook, Apple Calendar అన్నీ అదే వాడతాయి. **Interview lo "I'd follow the RRULE shape from iCalendar" అని చెప్పడం** – ఇది మీరు దీన్ని ఊహించలేదని, ఒక ప్రమాణాన్ని తెలుసునని చూపిస్తుంది.

11. మినహాయింపులు – "ఈ ఒక్క వారం వద్దు"

కానీ ఇప్పుడు ఒక కొత్త సమస్య: "వచ్చే సోమవారం **standup** రద్దు". ఒక నియమంలో ఒక్క రోజుని ఎలా తీసేయాలి?

జవాబు: **నియమం + మినహాయింపులు**. రెండు రకాల మినహాయింపులు కావాలి – రద్దు చేసినవి, మరియు మార్చినవి:

```

class RecurrenceWithExceptions extends Recurrence {
    constructor(opts) {
        super(opts);
        this.exceptions = new Set(); // రద్దు చేసిన రోజులు
        this.overrides = new Map(); // సమయం మార్చిన రోజులు
    }

    cancel(day) { this.exceptions.add(day); }
    reschedule(day, newStart) { this.overrides.set(day, newStart); }

    occurrencesIn(s, e, base) {
        return super.occurrencesIn(s, e, base)
            .filter(day => !this.exceptions.has(day)) // రద్దు తీసేయడం
            .map(day => ({ day, start: this.overrides.get(day) ?? 600 }));
    }
}

```

రోజులు 0–35: రోజు 0@10:00 రోజు 7@10:00 రోజు 21@11:00 రోజు 28@10:00 రోజు 35@10:00
 → రోజు 14 లేదు (రద్దు) • రోజు 21 కి 11:00 (మార్చబడింది)

→ నియమం 1 row, మినహాయింపులు 2 rows. విస్తరించిన 52 rows కాదు.

రోజు 14 జాబితాలో లేదు, మరియు రోజు 21 కి 11:00 – మిగతావన్నీ 10:00. మొత్తం నిల్వ: 1 నియమం + 2 మినహాయింపులు = 3 rows, 52 కాదు.

ఇదే నిజమైన calendar systems చేసేది. Google Calendar lo ఒక recurring meeting ని "ఈ ఒక్కసారి" మార్చినప్పుడు, అది ఆ ఒక్క సందర్భానికి ఒక **exception** సృష్టిస్తుంది – మొత్తం series ని విస్తరించదు.

Part 5 – పూర్తి system

12. Step – ఢీ కొంటే ఏం చేయాలి • ఒక మారే నియమం

ఇప్పటిదాకా ఢీ కొంటే **తిరస్కరించాం**. కానీ అది ఒక్కటే సరైన ప్రవర్తన కాదు:

- **తిరస్కరించడం** – సాధారణ నియమం
- **తోసేయడం** – CEO review ఒక coffee chat ని తోసేయొచ్చు
- **వేచి ఉంచడం** – గది ఖాళీ అయితే తెలియజేయడం
- **వేరే గది సూచించడం**

అంటే ఇది ఒక **మారే business నియమం**, మరియు మీకు ఈ కదలిక ఇప్పుడు అలవాటు – **Strategy**:

```
class RejectOnConflict {
  resolve(existing, wanted) {
    return { ok: false, reason: `CLASH: ${existing.title}` };
  }
}

class BumpLowerPriority {
  resolve(existing, wanted) {
    return wanted.priority > existing.priority
      ? { ok: true, bump: existing }
      : { ok: false, reason: `CLASH: ${existing.title} (ప్రాధాన్యత ఎక్కువ)` };
  }
}
```

ఇది ఒక సాంకేతిక నిర్ణయం కాదు

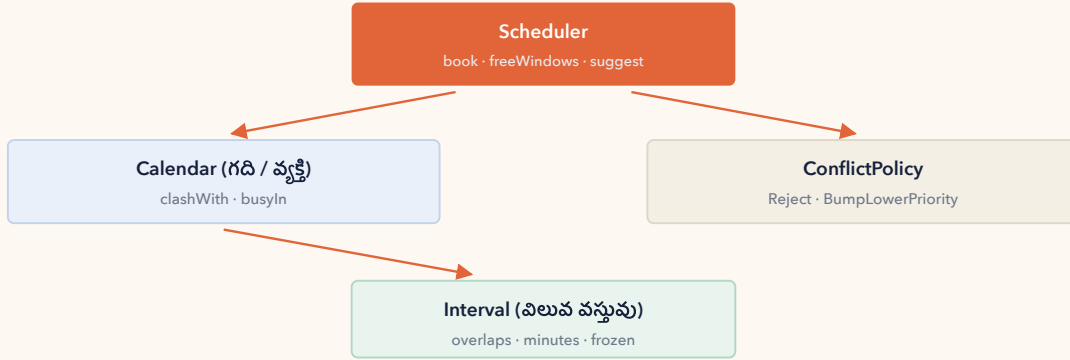
"CEO meeting ఒక team sync ని తోసేయొచ్చా?" – ఇది ఒక **సంస్థాగత** ప్రశ్న, ఒక engineering ప్రశ్న కాదు. మరియు ఒక్కో కంపెనీలో ఒక్కో జవాబు.

అలాంటి నియమాలని **ఎప్పుడూ if గా code lo పాతకూడదు** – అవి ఒక వస్తువుగా బయట ఉండాలి. §12 lo ఇది స్పష్టంగా కనిపిస్తోంది: కొత్త policy = కొత్త class, **Scheduler** lo సున్నా మార్పులు.

13. మొత్తం code ఒకే చోట · నడిపి చూద్దాం

నిర్మాణం · INTERVAL కేంద్రంలో

Overlap logic ఒక్కచోట - అదే ఈ design యొక్క గుండె



```
class Interval {
  constructor(start, end) {
    if (end <= start) throw new Error(`EMPTY_INTERVAL: ${hhmm(start)}-${hhmm(end)}`);
    Object.assign(this, { start, end }); Object.freeze(this);
  }
  overlaps(o) { return this.start < o.end && o.start < this.end; }
  get minutes() { return this.end - this.start; }
  toString() { return `${hhmm(this.start)}-${hhmm(this.end)}`; }
}

class RejectOnConflict {
  resolve(existing, wanted) {
    return { ok: false, reason: `CLASH: ${existing.title}` };
  }
}

class BumpLowerPriority {
  resolve(existing, wanted) {
    return wanted.priority > existing.priority
      ? { ok: true, bump: existing }
      : { ok: false, reason: `CLASH: ${existing.title} (ప్రాధాన్యత ఎక్కువ)` };
  }
}

class Booking {
  constructor(id, interval, title, organiser,
    { priority = 0, attendees = [] } = {}) {
    Object.assign(this, { id, interval, title, organiser, priority, attendees });
  }
}

class Calendar {
  #bookings = [];
  constructor(owner) { this.owner = owner; }
  add(b) { this.#bookings.push(b); }
  remove(id) {
    const i = this.#bookings.findIndex(b => b.id === id);
    return i === -1 ? null : this.#bookings.splice(i, 1)[0];
  }
  clashWith(interval) {
    return this.#bookings.find(b => b.interval.overlaps(interval)) ?? null;
  }
}
```

```

    }
    busyIn(dayStart, dayEnd) {
        return this.#bookings.map(b => b.interval)
            .filter(i => i.end > dayStart && i.start < dayEnd);
    }
    get bookings() { return [...this.#bookings]; }
}

class Scheduler {
    #seq = 0;
    constructor(rooms, { policy = new RejectOnConflict() } = {}) {
        this.rooms = new Map(rooms.map(r => [r, new Calendar(r)]));
        this.people = new Map();
        this.policy = policy;
    }
    // గది అయినా, వ్యక్తి అయినా - ఒకే విధంగా ఒక Calendar
    #cal(owner) {
        if (this.rooms.has(owner)) return this.rooms.get(owner);
        if (!this.people.has(owner)) this.people.set(owner, new Calendar(owner));
        return this.people.get(owner);
    }

    freeWindows(owners, dayStart, dayEnd) {
        const busy = owners
            .flatMap(o => this.#cal(o).busyIn(dayStart, dayEnd))
            .map(i => ({ start: Math.max(i.start, dayStart),
                end: Math.min(i.end, dayEnd) }))
            .sort((a, b) => a.start - b.start);

        const merged = [];
        for (const m of busy) {
            const last = merged[merged.length - 1];
            if (last && m.start <= last.end) last.end = Math.max(last.end, m.end);
            else merged.push({ ...m });
        }
        const free = []; let cursor = dayStart;
        for (const m of merged) {
            if (m.start > cursor) free.push(new Interval(cursor, m.start));
            cursor = Math.max(cursor, m.end);
        }
        if (cursor < dayEnd) free.push(new Interval(cursor, dayEnd));
        return free;
    }

    suggest(owners, durationMin, dayStart, dayEnd) {
        return this.freeWindows(owners, dayStart, dayEnd)
            .filter(w => w.minutes >= durationMin)
            .map(w => new Interval(w.start, w.start + durationMin));
    }

    book(room, attendees, interval, title, opts = {}) {
        const wanted = new Booking(`M${++this.#seq}`, interval, title,
            attendees[0], { ...opts, attendees });
        const owners = [room, ...attendees];
        for (const o of owners) { // గది మరియు ప్రతి attendee
            const clash = this.#cal(o).clashWith(interval);
            if (!clash) continue;
            const r = this.policy.resolve(clash, wanted);
            if (!r.ok) return { ok: false, where: o, reason: r.reason };
        }
    }
}

```

```

    for (const oo of owners) this.#cal(oo).remove(clash.id);
  }
  for (const o of owners) this.#cal(o).add(wanted);
  return { ok: true, booking: wanted };
}
}

```

```

--- Bookings ---
standup : true
వెనువెంటనే: true ← §4 lo ఇది విఫలమైంది
ఢీ కొట్టింది: { ok: false, where: 'Everest', reason: 'CLASH: standup' }
వేరే గది : true

--- ఖాళీ కిటికీలు ---
Everest : 11:00-13:00  14:30-16:00  17:00-18:00
asha+ravi: 11:00-13:00  14:30-16:00  17:00-18:00
45 నిమిష సూచనలు: 11:00-11:45  14:30-15:15  17:00-17:45

--- ప్రాధాన్యత policy ---
తక్కువ ప్రాధాన్యత: { ok: false, where: 'Board', reason: 'CLASH: team sync (ప్రాధాన్యత ఎక్కువ)' }
ఎక్కువ ప్రాధాన్యత: true
Board lo ఇప్పుడు: 10:30-11:30 CEO review

```

ఈ output ని పంక్తి పంక్తిగా చదువుదాం

వెనువెంటనే: true – §4 lo విఫలమైన అదే booking ఇప్పుడు విజయవంతం. మొదటి విరుపు సరిచేయబడింది.

where: 'Everest' – తిరస్కరణ ఎక్కడ ఢీ కొట్టిందో చెప్తోంది. గదా, ఒక attendee నా? UI "Everest is busy" vs "Ravi is busy" అని వేరుగా చూపించగలదు. **తిరస్కరణ కూడా ఒక API** – Deep Dive 03 §12 lo చూసిన అదే ఆలోచన.

Everest మరియు asha+ravi ఒక కిటికీలు – యాదృచ్ఛికం కాదు: ఈ ఉదాహరణలో అన్ని meetings Everest lo ఉన్నాయి, కాబట్టి గది యొక్క busy = ఆ ఇద్దరి busy కలయిక.

45 నిమిష సూచనలు – కిటికీలు లెక్కించాక, నిడివి ఒక సాధారణ filter. 11:00-13:00 అనే రెండు గంటల కిటికీ నుంచి 11:00-11:45 అనే ఒక సూచన.

ప్రాధాన్యత policy – coffee chat (priority 0) తిరస్కరించబడింది; CEO review (priority 5) team sync (priority 1) ని **తేసేసింది**. చివరి పంక్తి చూడండి: Board lo ఇప్పుడు CEO review **మాత్రమే** ఉంది. మరియు **Scheduler** lo ఒక్క అక్షరం కూడా మారలేదు – కేవలం ఒక constructor argument.

దశల నుంచి ఇక్కడికి – ఏమి చేరింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§3	Bookings జాబితా, overlap	మౌలిక అస్థిపంజరం
§4 (విరుపు)	Interval – అర్థ-తెరిచినది	<= వెనువెంటనే meetings ని ఆపింది

ఎక్కడ నుంచి	ఏమి చేరింది	ఎందుకు
§7 (విరుపు)	<code>freeWindows</code> – merge + gaps	Grid నకిలీలు ఇచ్చింది, నిజమైనవి మిస్ చేసింది
§9 (విరుపు)	<code>Recurrence</code> – నియమం, ఫలితం కాదు	52 rows vs 1
§11	<code>exceptions</code> , <code>overrides</code>	"ఈ ఒక్క వారం వద్దు"
§12	<code>ConflictPolicy</code>	థీ కొంటే ఏం చేయాలి – business నిర్ణయం
ఇక్కడ	<code>where</code> in rejection	UI కి ఏది busy అని తెలియాలి

Part 6 – Interview Io

14. ఇదంతా 45 నిమిషాల్లో ఎలా చెప్పాలి

45 నిమిషాల TIME BUDGET

Code తక్కువ – కానీ మూడు విరుపులను చూపించడానికి సమయం కావాలి

5 నిమి

Clarify – పునరావృత్తి? timezones? వెనువెంటనే?

8 నిమి – Interval

అర్థ-తెరిచినది · ఐదు సందర్భాలు గీయండి

13 నిమి – freeWindows

merge + gaps · ఇదే కేంద్రం

9 నిమి

Recurrence + exceptions

10 నిమి

Policy · timezones · scale

ఏమి తప్పక చెప్పాలి

1. అర్థ-తెరిచిన విరామాలు (§5) – మరియు ఎందుకు. ఐదు సందర్భాల పట్టికని గీయండి; అది 30 సెకన్లు, మరియు అది మీ మొదటి ముద్ర.
2. **Overlap logic** Interval Io ఉండాలి, Meeting Io కాదు (§5).
3. **Grid sampling** ఎందుకు తప్పు (§7) – ముఖ్యంగా మీస్ అయిన slot.
4. పునరావృత్తి = నియమం, ఫలితాలు కాదు (§9) – మరియు exceptions.
5. **Timezones** (§16) – మీరే లేవనెత్తండి.

ఏమి వదిలేయాలి

- **BumpLowerPriority** రాయొద్దు – "policy ఒక interface" అని చెప్పి, ఒకటి రాసి, రెండోదాన్ని ఒక వాక్యంలో వివరించండి.
- **RecurrenceWithExceptions** పూర్తిగా రాయొద్దు – exceptions Set మరియు overrides Map ఉన్నాయని చెప్తే చాలు.
- **suggest()** వదిలేయండి – అది freeWindows మీద ఒక filter, స్పష్టం.

15. నీటితో చెప్పాల్సిన English script

"Three questions up front. Should back-to-back meetings be allowed – ten to eleven and then eleven to twelve? Do you want recurring meetings? And is this single-timezone or global?

I'll start with the piece everything else rests on: the interval. The obvious overlap test is start-a-less-than-or-equal end-b and end-a greater-than-or-equal start-b, and that's wrong – it says ten-to-eleven clashes with eleven-to-twelve, which happens in every office every day. The fix is treating intervals as half-open: a meeting ending at eleven runs up to but not including eleven. Then both comparisons are strict and back-to-back works.

I'd put that in an Interval value object rather than on Meeting, because overlap is a property of intervals, not of meetings. Otherwise the same logic gets copied into Room, User and Resource, and when you fix the boundary bug in one place the others keep it.

For finding free time, the tempting approach is to walk the day in fifteen-minute steps and test each candidate slot. That's wrong in two ways. It reports overlapping near-duplicates – ten-thirty, ten-forty-five and eleven are three 'slots' describing one free hour. And worse, it misses real openings: if someone's free from 10:07 to 10:37 that's a genuine thirty-minute slot, but it never lands on the grid, so the system says there's nothing. Making the step finer finds it but multiplies the duplicate problem.

So I'd flip it – instead of searching for free time, compute it. Take everyone's busy intervals, sort, merge the overlapping ones, and the gaps between them are the free windows. That's exact, it's independent of any grid, and duration becomes a filter afterwards rather than a parameter to the search.

One detail there: in the merge step I do use less-than-or-equal, even though I just removed it from overlap. That's deliberate – it's asking whether two busy blocks touch and should become one, not whether they conflict. Same symbol, different question.

For recurring meetings, I wouldn't expand them into rows. A weekly standup for a year is fifty-two rows, changing the time means fifty-two updates, and 'repeats forever' has no row count at all. I'd store the rule – roughly the iCalendar RRULE shape – and compute occurrences for whatever range is being viewed. Cancelling one week is an exception set, and moving one week is an override map, which is exactly what Google Calendar does.

Last thing: what happens on a conflict isn't a technical decision. Reject, bump a lower-priority meeting, waitlist, suggest another room – that varies by company, so it goes behind a policy interface rather than an if statement."

16. Follow-ups – timezones, scale, notifications

Follow-up	జవాబు	మారే classes
"Buffer time (meetings మధ్య 10 నిమి)"	తనిఖీకి ముందు interval ని రెండు వైపులా పెంచడం	<code>book</code> lo ఒక పంక్తి
"Room capacity ప్రకారం సూచించాలి"	<code>suggest</code> కి ఒక room filter	+1 కొత్తది
"Waitlist"	కొత్త <code>WaitlistPolicy</code>	+1 కొత్తది, 0 edits
"Notifications"	Deep Dive 07 యొక్క event bus – <code>MEETING_BOOKED</code> publish	0 concepts
"పలు shifts / working hours"	<code>dayStart</code> / <code>dayEnd</code> ఇప్పటికే parameters	0
"Timezones"	కింద చూడండి	–
"10,000 rooms, లక్షల bookings"	కింద చూడండి	Storage layer

Timezones – దాచిన రాక్షసుడు

"నా code lo అంతా నిమిషాల్లో ఉంది, timezone లేకుండా. అది ఒక్క కార్యాలయానికి సరిపోతుంది, కానీ global team కి కాదు.

నియమం: లోపల అంతా **UTC**. Timezone అనేది కేవలం ఒక ప్రదర్శన విషయం – user కి చూపించేటప్పుడు, మరియు user నుంచి తీసుకునేటప్పుడు మాత్రమే మార్చాలి.

కానీ పునరావృత్తి దీన్ని విరగొడుతుంది. "ప్రతి సోమవారం ఉదయం 9 గంటలకి, Hyderabad సమయం" – daylight saving ఉన్న దేశంలోని ఒక attendee కి అది సంవత్సరంలో రెండుసార్లు **వేరే UTC సమయం** అవుతుంది. కాబట్టి పునరావృత్త నియమాన్ని UTC lo store చేయకూడదు – దాన్ని **స్థానిక సమయం + timezone వేరు** గా store చేసి, ప్రతి సందర్భానికీ UTC ని లెక్కించాలి.

అంటే: **ఒక్కసారి జరిగే meeting కి UTC సరైనది; పునరావృత్త నియమానికి స్థానిక సమయం సరైనది.** ఈ తేడా చాలా calendar bugs కి మూలం."

Scale

"ఇప్పుడు `clashWith` మొత్తం జాబితాని scan చేస్తుంది – $O(n)$. వంద bookings కి పర్వాలేదు.

పెద్దదైతే: bookings ని **సమయం ప్రకారం sorted** గా ఉంచి binary search – $O(\log n)$. ఇంకా పెద్దదైతే **interval tree**, అది "ఈ పరిధితో కలిసేవి ఏవి?" అనే ప్రశ్నకి సరిగ్గా సరిపోయేది.

కానీ ఆచరణలో ఒక సులభమైన ఉపాయం ఎక్కువ పనిచేస్తుంది: **రోజు వారీగా విభజించడం**. ఒక గది యొక్క ఒక రోజు bookings ఎప్పుడూ తక్కువ (20?), కాబట్టి ఆ రోజుని తీసుకుని linear scan చేయడం సరిపోతుంది. **Interval tree కంటే ముందు ఈ విభజనని ప్రస్తావించడం** ఆచరణాత్మక ఆలోచనని చూపిస్తుంది."

17. ఏమి నేర్చుకున్నాం

అలోచన	ఇక్కడ ఎలా కనిపించింది	ఇంకెక్కడ వస్తుంది
సరిహద్దులు · <code><</code> నా <code><=</code> నా	వెనువెంటనే meetings (§4)	Pagination, ranges, binary search, versions
అర్థ-తెరిచిన విరామాలు	<code>[start, end)</code> (§5)	Array slices, date ranges, price bands
Logic విలువ వస్తువులో	<code>Interval.overlaps</code> (§5)	Money, Duration, Range – ఏ value type lo అయినా
వెతకడం కాదు, లెక్కించడం	Grid → merge+gaps (§8)	Deep Dive 08 §5 – అదే అలోచన
నియమాన్ని store చెయ్యి, ఫలితాలని కాదు	Recurrence (§9)	Deep Dive 05 – ఖర్చులు vs అప్పులు
నియమం + మినహాయింపులు	<code>exceptions</code> , <code>overrides</code> (§11)	Feature flags, pricing rules, permissions
తిరస్కరణ ఒక API	<code>where</code> (§13)	Deep Dive 03 §12 – 429 + Retry-After

ఇక్కడి నుంచి ఎక్కడికి

ఈ series lo ఇప్పటివరకు: **01 Parking Lot · 02 Cache · 03 Rate Limiter · 04 BookMyShow · 05 Splitwise · 06 Elevator · 07 Pub-Sub · 08 HashMap · 09 Chess · 10 Meeting Scheduler.**

పదే doc వద్ద ఒక నమూనా స్పష్టంగా కనిపిస్తోంది. "నియమాన్ని store చెయ్యి, ఫలితాలని కాదు" ఇప్పుడు రెండోసారి వచ్చింది (05 మరియు ఇక్కడ). "మారే business నియమాన్ని బయట పెట్టు" ఆరోసారి. "తిరస్కరణ ఒక API" మూడోసారి.

LLD interviews కొత్త అలోచనలని పరీక్షించవు – ఇవే కొన్ని అలోచనలని వేర్వేరు వేషాల్లో పరీక్షిస్తాయి. అందుకే పది problems లోతుగా చేస్తే, పదికొండేది మీకు ఇప్పటికే తెలిసినట్టు అనిపిస్తుంది.

వేగవంతమైన revision కోసం – [LLD_Design_Problems_Telugu.pdf](#) lo Problem 15.

Meeting Scheduler – అడుగు అడుగునా · ఈ doc lo ఉన్న ప్రతి output నిజంగా `node` lo run చేసి తీసినదే 